LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5

Connect to the Internet

Oleh:

Akhmad Chaidar Ananda **NIM. 2310817110015**

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Akhmad Chaidar Ananda
NIM : 2310817110015

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Zulfa Auliya Akbar NIM. 2210817210026	Muti`a Maulida S.Kom M.T.I NIP. 19881027 201903 20 13

DAFTAR ISI

DAFTAR GAMBAR

DAFTAR TABEL

SOAL 1

Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

a. Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
b. Gunakan KotlinX Serialization sebagai library JSON
c. Gunakan library seperti Coil atau Glide untuk image loading.
d. API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: https://developer.themoviedb.org/docs/getting-started
e. Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
f. Gunakan caching strategy pada Room..
g. Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

1. item_row_morty.xml

Tabel 1. Source Code Jawaban Soal 1

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayou
3  t
4  xmlns:android="http://schemas.android.com/apk/res
5  /android"
6
7  xmlns:tools="http://schemas.android.com/tools"
8      android:layout_width="match_parent"
```

```
9       android:layout_height="wrap_content"
10
11  xmlns:app="http://schemas.android.com/apk/res-
12  auto"
13      android:padding="20dp">
14
15      <androidx.cardview.widget.CardView
16          android:id="@+id/cv_img"
17          android:layout_width="wrap_content"
18          android:layout_height="wrap_content"
19          app:cardBackgroundColor="@color/teal"
20          app:cardCornerRadius="10dp"
            app:layout_constraintTop_toTopOf="parent"

    app:layout_constraintBottom_toBottomOf="parent"

    app:layout_constraintStart_toStartOf="parent">

            <ImageView
                android:id="@+id/item_image_morty"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"

    android:background="@drawable/ic_launcher_backgro
    und" />

        </androidx.cardview.widget.CardView>

        <TextView
            android:id="@+id/item_name_morty"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
```

```
        android:text="Name"
        android:textSize="25sp"
        android:textStyle="bold"

app:layout_constraintStart_toEndOf="@id/cv_img"
        android:layout_marginStart="10dp"
        />

    <TextView
        android:id="@+id/item_species_morty"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Species: Human"
        android:textSize="16sp"

app:layout_constraintStart_toEndOf="@id/cv_img"

app:layout_constraintTop_toBottomOf="@id/item_nam
e_morty"
        android:layout_marginStart="10dp"
        android:layout_marginTop="5dp"
        />

    <TextView
        android:id="@+id/item_alive_morty"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Alive"
        android:textSize="16sp"

app:layout_constraintStart_toEndOf="@id/cv_img"
```

```
app:layout_constraintTop_toBottomOf="@id/item_species_morty"
        android:layout_marginStart="10dp"
        android:layout_marginTop="5dp"
        />

    <LinearLayout
        android:id="@+id/button_container"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center"
        android:layout_marginTop="10dp"

app:layout_constraintTop_toBottomOf="@id/item_alive_morty"

app:layout_constraintStart_toEndOf="@id/cv_img"
        app:layout_constraintEnd_toEndOf="parent"
        >


<com.google.android.material.button.MaterialButton
            android:id="@+id/btn_details"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Detail"
            android:textSize="12sp"
            android:textAllCaps="false"
```

```
android:textColor="@android:color/white"

android:background="@drawable/rounded_button_prim
ary"
            android:layout_marginEnd="4dp"/>


<com.google.android.material.button.MaterialButto
n
            android:id="@+id/btn_wiki"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Wiki"
            android:textSize="12sp"
            android:textAllCaps="false"

android:textColor="@android:color/white"

android:background="@drawable/rounded_button_prim
ary"
            android:layout_marginEnd="4dp"/>


<com.google.android.material.button.MaterialButto
n
            android:id="@+id/btn_favorite"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Fav"
            android:textSize="12sp"
```

```
            android:textAllCaps="false"

android:textColor="@android:color/white"

android:background="@drawable/rounded_button_prim
ary" />
    </LinearLayout>

</androidx.constraintlayout.widget.ConstraintLayo
ut>
```

2. fragment_home.xml

Tabel 2. Source Code Jawaban Soal 1

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayou
3  t
4  xmlns:android="http://schemas.android.com/apk/res
5  /android"
6
7  xmlns:tools="http://schemas.android.com/tools"
8      android:layout_width="match_parent"
9      android:layout_height="match_parent"
10
11 xmlns:app="http://schemas.android.com/apk/res-
12 auto"
13
14 tools:context=".presentation.home.HomeFragment"
15     android:orientation="vertical">
16
17     <androidx.recyclerview.widget.RecyclerView
```

```
18          android:id="@+id/rv_morty"
19          android:layout_width="0dp"
20          android:layout_height="0dp"

    app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"

    app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            tools:listitem="@layout/item_row_morty">

        </androidx.recyclerview.widget.RecyclerView>

        <ProgressBar
            android:id="@+id/progress_bar"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"

    app:layout_constraintBottom_toBottomOf="parent"
            app:layout_constraintEnd_toEndOf="parent"

    app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintTop_toTopOf="parent"
            />

        <TextView
            android:id="@+id/tvError"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Terjadi kesalahan"
            android:visibility="gone"
```

```
    android:textColor="@android:color/holo_red_dark"
            android:layout_gravity="center" />


</androidx.constraintlayout.widget.ConstraintLayo
ut>
```

3. fragment_detail.xml

Tabel 3. Source Code Jawaban Soal 1

```
1   <?xml version="1.0" encoding="utf-8"?>
2   <androidx.constraintlayout.widget.ConstraintLayou
3   t
4   xmlns:android="http://schemas.android.com/apk/res
5   /android"
6
7   xmlns:tools="http://schemas.android.com/tools"
8       android:layout_width="match_parent"
9       android:layout_height="match_parent"
10
11  xmlns:app="http://schemas.android.com/apk/res-
12  auto"
13
14  tools:context=".presentation.detail.DetailFragmen
15  t">
16
17      <ImageView
18          android:id="@+id/img_item_photo"
19          android:layout_width="120dp"
20          android:layout_height="120dp"
            android:layout_marginTop="84dp"
            android:scaleType="centerCrop"
            app:layout_constraintEnd_toEndOf="parent"
```

```
app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"

android:src="@drawable/ic_launcher_background"
        />

    <TextView
        android:id="@+id/tv_name"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"

app:layout_constraintTop_toBottomOf="@id/img_item
_photo"

app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:text="Nama Char"
        android:textSize="20sp"
        android:textStyle="bold"
        android:textColor="@color/black"
        />

    <TextView
        android:id="@+id/tv_species"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Species Char"
        android:textStyle="bold"
        android:textColor="@color/black"

app:layout_constraintTop_toBottomOf="@id/tv_name"
```

```
app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        />

    <TextView
        android:id="@+id/tv_status"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Status Char"
        android:textStyle="bold"
        android:textColor="@color/black"

app:layout_constraintTop_toBottomOf="@id/tv_speci
es"

app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        />

</androidx.constraintlayout.widget.ConstraintLayo
ut>
```

4. item_row_fav.xml

Tabel 4. Source Code Jawaban Soal 1

```
1   <?xml version="1.0" encoding="utf-8"?>
2   <androidx.constraintlayout.widget.ConstraintLayou
3   t
4   xmlns:android="http://schemas.android.com/apk/res
5   /android"
6
7   xmlns:tools="http://schemas.android.com/tools"
8       android:layout_width="match_parent"
9       android:layout_height="wrap_content"
10
11  xmlns:app="http://schemas.android.com/apk/res-
12  auto"
13      android:padding="20dp">
14
15      <androidx.cardview.widget.CardView
16          android:id="@+id/cv_img_fav"
17          android:layout_width="wrap_content"
18          android:layout_height="wrap_content"
19          app:cardBackgroundColor="@color/teal"
20          app:cardCornerRadius="10dp"
            app:layout_constraintTop_toTopOf="parent"

    app:layout_constraintBottom_toBottomOf="parent"

    app:layout_constraintStart_toStartOf="parent">

            <ImageView
                android:id="@+id/item_image_fav"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
```

```
android:background="@drawable/ic_launcher_backgro
und" />

    </androidx.cardview.widget.CardView>

    <TextView
        android:id="@+id/item_name_fav"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Name"
        android:textSize="25sp"
        android:textStyle="bold"

app:layout_constraintStart_toEndOf="@id/cv_img_fa
v"
        android:layout_marginStart="10dp"
        />

    <TextView
        android:id="@+id/item_species_fav"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Species: Human"
        android:textSize="16sp"

app:layout_constraintStart_toEndOf="@id/cv_img_fa
v"

app:layout_constraintTop_toBottomOf="@id/item_nam
e_fav"
        android:layout_marginStart="10dp"
```

```
        android:layout_marginTop="5dp"
        />

    <TextView
        android:id="@+id/item_alive_fav"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Alive"
        android:textSize="16sp"

app:layout_constraintStart_toEndOf="@id/cv_img_fa
v"

app:layout_constraintTop_toBottomOf="@id/item_spe
cies_fav"
        android:layout_marginStart="10dp"
        android:layout_marginTop="5dp"
        />


<com.google.android.material.button.MaterialButto
n
        android:id="@+id/btn_remove_fav"
        android:layout_width="0dp"
        android:layout_height="wrap_content"

app:layout_constraintTop_toBottomOf="@id/item_ali
ve_fav"

app:layout_constraintStart_toEndOf="@id/cv_img_fa
v"
        android:layout_weight="1"
```

```
        android:layout_marginStart="10dp"
        android:layout_marginTop="5dp"
        android:text="Remove"
        android:textAllCaps="false"
        android:textColor="@android:color/white"
        android:drawablePadding="4dp"
        android:elevation="2dp"

android:background="@drawable/rounded_button_prim
ary"
        />

</androidx.constraintlayout.widget.ConstraintLayo
ut>
```

5. fragment_favorite.xml

Tabel 5. Source Code Jawaban Soal 1

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayou
3  t
4  xmlns:android="http://schemas.android.com/apk/res
5  /android"
6
7  xmlns:tools="http://schemas.android.com/tools"
8      android:layout_width="match_parent"
9      android:layout_height="match_parent"
10
11 xmlns:app="http://schemas.android.com/apk/res-
12 auto"
13
14 tools:context=".presentation.favorites.FavoriteFr
15 agment">
```

```
16
17      <androidx.recyclerview.widget.RecyclerView
18          android:id="@+id/rvFavorite"
19          android:layout_width="0dp"
20          android:layout_height="0dp"
            android:padding="8dp"

    app:layoutManager="androidx.recyclerview.widget.L
    inearLayoutManager"
            app:layout_constraintTop_toTopOf="parent"

    app:layout_constraintBottom_toBottomOf="parent"

    app:layout_constraintStart_toStartOf="parent"
            app:layout_constraintEnd_toEndOf="parent"

    tools:listitem="@layout/item_row_favorite"
            />

    </androidx.constraintlayout.widget.ConstraintLayo
    ut>
```

6. bottom_navigation_menu.xml

Tabel 6. Source Code Jawaban Soal 1

```
1   <?xml version="1.0" encoding="utf-8"?>
2   <menu
3   xmlns:android="http://schemas.android.com/apk/res
4   /android">
5       <item
6           android:id="@+id/navigation_home"
7           android:title="Home"
8           android:icon="@drawable/ic_home_24"/>
```

```
9
10      <item
11          android:id="@+id/navigation_fav"
12          android:title="Favorites "
13          android:icon="@drawable/ic_bookmark_24"
14          />
15  </menu>
```

7. activity_main.xml

Tabel 7. Source Code Jawaban Soal 1

```
1   <?xml version="1.0" encoding="utf-8"?>
2   <RelativeLayout
3   xmlns:android="http://schemas.android.com/apk/res
4   /android"
5
6   xmlns:app="http://schemas.android.com/apk/res-
7   auto"
8
9   xmlns:tools="http://schemas.android.com/tools"
10      android:id="@+id/main"
11      android:layout_width="match_parent"
12      android:layout_height="match_parent"
13      tools:context=".presentation.MainActivity"
14      android:orientation="vertical">
15
16      <FrameLayout
17          android:id="@+id/navigation_content"
18          android:layout_width="match_parent"
19          android:layout_height="match_parent"/>
20


    <com.google.android.material.bottomnavigation.Bot
```

```
tomNavigationView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:id="@+id/btn_navigation"
        app:menu="@menu/bottom_navigation_menu"
        android:layout_alignParentBottom="true"/>


</RelativeLayout>
```

8. data/model/CharacterDto.kt

Tabel 8. Source Code Jawaban Soal 1

```
1  package com.example.rickandmortyapi.data.model
2
3  import kotlinx.serialization.Serializable
4
5  @Serializable
6  data class CharacterDto(
7      val id: Int,
8      val name: String,
9      val status: String,
10     val species: String,
11     val image: String
   )
```

9. data/model/CharacterResponse.kt

Tabel 9. Source Code Jawaban Soal 1

```
1  import
2  com.example.rickandmortyapi.data.model.CharacterD
3  to
4  import kotlinx.serialization.Serializable
5
```

```
@Serializable
data class CharacterResponse(
    val results: List<CharacterDto>
)

@Serializable
data class Character(
    val id: Int,
    val name: String,
    val status: String,
    val species: String,
    val image: String,
    val origin: Origin,
    val location: Location
)

@Serializable
data class Origin(
    val name: String,
    val url: String
)

@Serializable
data class Location(
    val name: String,
    val url: String
)
```

10. data/Api/ApiService.kt

Tabel 10. Source Code Jawaban Soal 1

```
package com.example.rickandmortyapi.data.Api

```

```
3  import CharacterResponse
4  import retrofit2.http.GET
5
6  interface ApiService {
7      @GET("character")
8      suspend fun getMorty(): CharacterResponse
9  }
```

11. data/Api/ApiConfig.kt

Tabel 11. Source Code Jawaban Soal

```
1  package com.example.rickandmortyapi.data.Api
2
3  import
4  com.jakewharton.retrofit2.converter.kotlinx.seria
5  lization.asConverterFactory
6  import kotlinx.serialization.json.Json
7  import okhttp3.MediaType.Companion.toMediaType
8  import retrofit2.Retrofit
9
10 object ApiConfig {
11     const val baseUrl =
12 "https://rickandmortyapi.com/api/"
13     val contentType =
14 "application/json".toMediaType()
15
16     fun getApiService(): ApiService {
17         return Retrofit.Builder()
18             .baseUrl(baseUrl)
19             .addConverterFactory(Json {
20 ignoreUnknownKeys = true
   }.asConverterFactory(contentType))
```

```
            .build()
            .create(ApiService::class.java)
    }
}
```

12. data/model/CharacterEntity.kt

Tabel 12. Source Code Jawaban Soal 1

```
1  package com.example.rickandmortyapi.data.model
2
3  import androidx.room.Entity
4  import androidx.room.PrimaryKey
5
6  @Entity(tableName = "favorites")
7  data class CharacterEntity(
8      @PrimaryKey val id: Int,
9      val name: String,
10     val species: String,
11     val status: String,
12     val image: String
13 )
```

13. data/db/CharacterDao.kt

Tabel 13. Source Code Jawaban Soal 1

```
1  package com.example.rickandmortyapi.data.db
2
3  import androidx.lifecycle.LiveData
4  import androidx.room.Dao
5  import androidx.room.Delete
6  import androidx.room.Insert
7  import androidx.room.OnConflictStrategy
```

```
import androidx.room.Query
import
com.example.rickandmortyapi.data.model.CharacterE
ntity

@Dao
interface CharacterDao {
    @Query("SELECT * FROM favorites")
    fun getFavorites():
LiveData<List<CharacterEntity>>

    @Insert(onConflict =
OnConflictStrategy.REPLACE)
    suspend fun addToFavorite (character:
CharacterEntity)

    @Delete
    suspend fun removeFavorite (character:
CharacterEntity)
}
```

14. data/db/AppDatabase.kt

Tabel 14. Source Code Jawaban Soal 1

```
package com.example.rickandmortyapi.data.db

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
import
com.example.rickandmortyapi.data.model.CharacterE
ntity
```

```
10
11  @Database(entities = [CharacterEntity::class],
12  version = 1, exportSchema = false)
13  abstract class AppDatabase: RoomDatabase() {
14      abstract fun characterDao(): CharacterDao
15
16      companion object {
17          @Volatile
18          private var instance: AppDatabase? = null
19
20          fun getDatabase (context: Context):
    AppDatabase =
              instance ?: synchronized(this) {
                  instance ?: Room.databaseBuilder(
                      context.applicationContext,
                      AppDatabase::class.java,
    "character_db"
                  ).build().also { instance = it }
              }
      }
    }
```

15. data/repository/CharacterRepository.kt

Tabel 15. Source Code Jawaban Soal 1

```
1  package
2  com.example.rickandmortyapi.data.repository
3
4  import androidx.lifecycle.LiveData
5  import
6  com.example.rickandmortyapi.data.Api.ApiService
7  import
8  com.example.rickandmortyapi.data.db.CharacterDao
```

```
import
com.example.rickandmortyapi.data.model.CharacterDto
import
com.example.rickandmortyapi.data.model.CharacterEntity

class CharacterRepository(
    private val api: ApiService,
    private val dao: CharacterDao
) {
    val favorites: LiveData<List<CharacterEntity>> = dao.getFavorites()

    suspend fun fetchCharacters(): List<CharacterDto> = api.getMorty().results

    suspend fun addToFavorite(dto: CharacterDto) {
        dao.addToFavorite(CharacterEntity(dto.id, dto.name, dto.species, dto.status, dto.image))
    }

    suspend fun removeFavorite(entity: CharacterEntity) {
        dao.removeFavorite(entity)
    }
}
```

16. presentation/adapter/CharacterAdapter.kt

Tabel 16. Source Code Jawaban Soal 1

```
1  package
2  com.example.rickandmortyapi.presentation.adapter
3
4  import android.annotation.SuppressLint
5  import android.view.LayoutInflater
6  import android.view.ViewGroup
7  import android.widget.Toast
8  import androidx.recyclerview.widget.RecyclerView
9  import com.bumptech.glide.Glide
10 import com.example.rickandmortyapi.R
11 import
12 com.example.rickandmortyapi.data.model.CharacterD
13 to
14 import
15 com.example.rickandmortyapi.databinding.ItemRowMo
16 rtyBinding
17
18 class CharacterAdapter (
19     private val dataMorty:
20 ArrayList<CharacterDto>,
    private val onDetailsClick: (String, String,
    String, String) -> Unit,
    private val onWikisClick: (String) -> Unit,
    private val onFavoriteClick: (CharacterDto) -
    > Unit
    ) :
    RecyclerView.Adapter<CharacterAdapter.MyViewHolde
    r>() {


    inner class MyViewHolder (val binding:
```

```
ItemRowMortyBinding) :
RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent:
ViewGroup, viewType: Int): MyViewHolder {
        val binding =
ItemRowMortyBinding.inflate(LayoutInflater.from(p
arent.context), parent, false)
        return MyViewHolder(binding)
    }

    override fun getItemCount(): Int =
dataMorty.size

    override fun onBindViewHolder(holder:
MyViewHolder, position: Int) {
        val item = dataMorty[position]

        holder.binding.apply {
            itemNameMorty.text = item.name
            itemSpeciesMorty.text = item.species
            itemAliveMorty.text = item.status

            Glide.with(itemImageMorty)
                .load(item.image)

.error(R.drawable.ic_launcher_background)
                .into(itemImageMorty)

            btnDetails.setOnClickListener {
                onDetailsClick(item.image,
item.name, item.species, item.status)
```

```
            }

            btnWiki.setOnClickListener {
                onWikisClick(item.name ?: "")
            }

            btnFavorite.setOnClickListener {
                onFavoriteClick(item)
            }

            holder.itemView.setOnClickListener {

Toast.makeText(holder.itemView.context,
item.name, Toast.LENGTH_SHORT).show()
            }
        }
    }

    @SuppressLint("NotifyDataSetChanged")
    fun updateData (newData: List<CharacterDto>)
{
        if (newData.isNotEmpty()) {
            dataMorty.clear()
            dataMorty.addAll(newData)
            notifyDataSetChanged()
        }

    }
}
```

17. data/viewmodel/CharacterViewModel.kt

Tabel 17. Source Code Jawaban Soal 1

```
1   package
2   com.example.rickandmortyapi.data.viewmodel
3
4   import android.app.Application
5   import androidx.lifecycle.AndroidViewModel
6   import androidx.lifecycle.LiveData
7   import androidx.lifecycle.MutableLiveData
8   import androidx.lifecycle.viewModelScope
9   import
10  com.example.rickandmortyapi.data.Api.ApiConfig
11  import
12  com.example.rickandmortyapi.data.db.AppDatabase
13  import
14  com.example.rickandmortyapi.data.model.CharacterD
15  to
16  import
17  com.example.rickandmortyapi.data.model.CharacterE
18  ntity
19  import
20  com.example.rickandmortyapi.data.repository.Chara
    cterRepository
    import kotlinx.coroutines.launch


    class CharacterViewModel(application:
    Application): AndroidViewModel(application) {
        private val repository: CharacterRepository


        private val _characters =
    MutableLiveData<List<CharacterDto>>()
        val characters: LiveData<List<CharacterDto>>
```

```
get() = _characters
    val favorites:
LiveData<List<CharacterEntity>>

    init {
        val db =
AppDatabase.getDatabase(application)
        repository =
CharacterRepository(ApiConfig.getApiService(),
db.characterDao())
        favorites = repository.favorites
        fetchCharacters()
    }

    private fun fetchCharacters() {
        viewModelScope.launch {
            _characters.value =
repository.fetchCharacters()
        }
    }

    fun addFavorite(dto: CharacterDto) {
        viewModelScope.launch {
            repository.addToFavorite(dto)
        }
    }

    fun removeFavorite(entity: CharacterEntity) {
        viewModelScope.launch {
            repository.removeFavorite(entity)
        }
```

```
    }
}
```

18. data/repository/CharacterRepositoryInstance.kt

Tabel 18. Source Code Jawaban Soal 1

```
1  package
2  com.example.rickandmortyapi.data.repository
3
4  import android.content.Context
5  import
6  com.example.rickandmortyapi.data.Api.ApiConfig
7  import
8  com.example.rickandmortyapi.data.db.AppDatabase
9
10 object CharacterRepositoryInstance {
11     fun getRepository(context: Context):
12 CharacterRepository {
13         val database =
14 AppDatabase.getDatabase(context)
15         val apiService =
16 ApiConfig.getApiService()
17         return CharacterRepository(apiService,
18 database.characterDao())
19     }
20 }
```

19. data/Resource.kt

Tabel 19. Source Code Jawaban Soal 1

```
1  package com.example.rickandmortyapi.data
2
3  sealed class Resource<T> (
```

```
4       val data: T? = null,
5       val message: String? = null
6   ) {
7       class Success<T>(data: T): Resource<T>(data)
8       class Loading<T>: Resource<T>()
9       class Error<T>(message: String, data: T? =
10  null): Resource<T>(data, message)
11  }
```

20. presentation/utils/HomeViewModelFactory.kt

Tabel 20. Source Code Jawaban Soal 1

```
1   package com.example.rickandmortyapi.utils
2
3   import HomeViewModel
4   import androidx.lifecycle.ViewModel
5   import androidx.lifecycle.ViewModelProvider
6   import
7   com.example.rickandmortyapi.data.repository.Chara
8   cterRepository
9
10  class ViewModelFactory(private val repository:
11  CharacterRepository): ViewModelProvider.Factory {
12      @Suppress("UNCHECKED_CAST")
13      override fun <T : ViewModel>
14  create(modelClass: Class<T>): T {
15          if
16  (modelClass.isAssignableFrom(HomeViewModel::class
17  .java)) {
18              return HomeViewModel(repository) as T
19          }
20          throw IllegalArgumentException("Unknown
    ViewModel Class")
```

```
    }
}
```

21. presentation/utils/Injection.kt

Tabel 21. Source Code Jawaban Soal 1

```
1  package com.example.rickandmortyapi.utils
2
3  import android.content.Context
4  import
5  com.example.rickandmortyapi.data.Api.ApiConfig
6  import
7  com.example.rickandmortyapi.data.db.AppDatabase
8  import
9  com.example.rickandmortyapi.data.repository.Chara
10 cterRepository
11
12 object Injection {
13     fun provideRepository(context: Context):
14 CharacterRepository {
15         val database =
16 AppDatabase.getDatabase(context)
17         val dao = database.characterDao()
18         val apiService =
19 ApiConfig.getApiService()
20         return CharacterRepository(apiService,
   dao)
   }
   }
```

22. presentation/home/HomeViewModel.kt

Tabel 22. Source Code Jawaban Soal 1

```
1  import android.content.res.Resources
2  import androidx.lifecycle.ViewModel
3  import androidx.lifecycle.viewModelScope
4  import com.example.rickandmortyapi.data.Resource
5  import
6  com.example.rickandmortyapi.data.model.CharacterD
7  to
8  import
9  com.example.rickandmortyapi.data.repository.Chara
10 cterRepository
11 import kotlinx.coroutines.flow.MutableStateFlow
12 import kotlinx.coroutines.flow.StateFlow
13 import kotlinx.coroutines.flow.asStateFlow
14 import kotlinx.coroutines.flow.collectLatest
15 import kotlinx.coroutines.launch
16
17 class HomeViewModel(private val repository:
18 CharacterRepository): ViewModel() {
19     private val _characterState =
20 MutableStateFlow<Resource<List<CharacterDto>>>(Re
   source.Loading())
   val characterState:
   StateFlow<Resource<List<CharacterDto>>> =
   _characterState.asStateFlow()

   init {
       getCharacters()
   }

   fun getCharacters () {
```

```
        viewModelScope.launch {
            try {
                _characterState.value =
Resource.Loading()
                val result =
repository.fetchCharacters()

                // mapping Dto
                val mapped = result.map {
                    CharacterDto(
                        id = it.id,
                        name = it.name,
                        species = it.species,
                        status = it.status,
                        image = it.image
                    )
                }

                _characterState.value =
Resource.Success(mapped)
            } catch (e: Exception) {
                _characterState.value =
Resource.Error(e.message ?: "Unknown Error")
            }
        }
    }

    fun addFavorite (character: CharacterDto) {
        viewModelScope.launch {
            repository.addToFavorite(character)
        }
```

```
    }
}
```

23. presentation/home/HomeFragment.kt

Tabel 23. Source Code Jawaban Soal 1

```
1  package
2  com.example.rickandmortyapi.presentation.home
3
4  import HomeViewModel
5  import android.content.Intent
6  import android.net.Uri
7  import android.os.Bundle
8  import android.util.Log
9  import androidx.fragment.app.Fragment
10 import android.view.LayoutInflater
11 import android.view.View
12 import android.view.ViewGroup
13 import android.widget.Toast
14 import androidx.lifecycle.ViewModelProvider
15 import androidx.lifecycle.lifecycleScope
16 import
17 androidx.recyclerview.widget.LinearLayoutManager
18 import com.example.rickandmortyapi.R
19 import com.example.rickandmortyapi.data.Resource
20 import
   com.example.rickandmortyapi.data.repository.Chara
   cterRepositoryInstance
   import
   com.example.rickandmortyapi.databinding.FragmentH
   omeBinding
   import
   com.example.rickandmortyapi.presentation.adapter.
```

```
CharacterAdapter
import
com.example.rickandmortyapi.presentation.detail.D
etailFragment
import
com.example.rickandmortyapi.utils.ViewModelFactor
y
import kotlinx.coroutines.flow.collectLatest
import kotlinx.coroutines.launch

class HomeFragment : Fragment() {
    private var _binding: FragmentHomeBinding? =
null
    private val binding get() = _binding!!

    private lateinit var mortyAdapter:
CharacterAdapter
    private lateinit var viewModel: HomeViewModel

    override fun onCreateView(
        inflater: LayoutInflater, container:
ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        _binding =
FragmentHomeBinding.inflate(inflater, container,
false)
        return binding.root
    }

    override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
```

```
        val repository =
CharacterRepositoryInstance.getRepository(require
Context())
        val factory =
ViewModelFactory(repository)
        viewModel = ViewModelProvider(this,
factory)[HomeViewModel::class.java]

        setupRecyclerView()
        observeViewModel()
    }

    private fun setupRecyclerView () {
        mortyAdapter = CharacterAdapter(
            arrayListOf(),
            onDetailsClick = { photo, name,
species, status ->
                val detailFragment =
DetailFragment().apply {
                    Log.e("Move to detail page",
"move to $name detail page")

                    arguments = Bundle().apply {
                        putString("EXTRA_NAME",
name)

putString("EXTRA_SPECIES", species)
                        putString("EXTRA_STATUS",
status)
                        putString("EXTRA_PHOTO",
photo)
                    }
```

```
                }


parentFragmentManager.beginTransaction()

.replace(R.id.navigation_content, detailFragment)
                    .addToBackStack(null)
                    .commit()
            },
            onWikisClick = { name ->
                val query = name.replace(" ",
"_")
                val wikiUrl =
"https://rickandmorty.fandom.com/wiki/$query"
                val intent =
Intent(Intent.ACTION_VIEW, Uri.parse(wikiUrl))
                startActivity(intent)
            },
            onFavoriteClick = { character ->
                viewModel.addFavorite(character)
                Toast.makeText(requireContext(),
"${character.name} ditambahkan ke favorit",
Toast.LENGTH_SHORT).show()
            }
        )

        binding.rvMorty.apply {
            layoutManager =
LinearLayoutManager(context)
            adapter = mortyAdapter
            setHasFixedSize(true)
        }
```

```
    }

    private fun observeViewModel() {
        viewLifecycleOwner.lifecycleScope.launch
{

viewModel.characterState.collectLatest { result -
>
                when (result) {
                    is Resource.Loading -> {

binding.progressBar.visibility = View.VISIBLE

binding.rvMorty.visibility = View.GONE

binding.tvError.visibility = View.GONE
                    }
                    is Resource.Success -> {

binding.progressBar.visibility = View.GONE

binding.rvMorty.visibility = View.VISIBLE

binding.tvError.visibility = View.GONE

mortyAdapter.updateData(result.data ?:
emptyList())
                    }
                    is Resource.Error -> {

binding.progressBar.visibility = View.GONE
```

```
    binding.rvMorty.visibility = View.GONE

    binding.tvError.visibility = View.VISIBLE

    Toast.makeText(requireContext(), result.message
    ?: "Unknown Error", Toast.LENGTH_LONG).show()
                        }
                    }
                }
            }
        }
```

24. presentation/detail/DetailFragment.kt

Tabel 24. Source Code Jawaban Soal 1

```
1   package
2   com.example.rickandmortyapi.presentation.detail
3
4   import android.os.Bundle
5   import androidx.fragment.app.Fragment
6   import android.view.LayoutInflater
7   import android.view.View
8   import android.view.ViewGroup
9   import com.bumptech.glide.Glide
10  import
11  com.example.rickandmortyapi.databinding.FragmentD
12  etailBinding
```

```
13
14  class DetailFragment : Fragment() {
15      private var _binding: FragmentDetailBinding?
16  = null
17      private val binding get() = _binding!!
18
19      override fun onCreateView(
20          inflater: LayoutInflater, container:
    ViewGroup?,
          savedInstanceState: Bundle?
      ): View? {
          _binding =
    FragmentDetailBinding.inflate(inflater,
    container, false)

          val name =
    arguments?.getString("EXTRA_NAME")
          val species =
    arguments?.getString("EXTRA_SPECIES")
          val status =
    arguments?.getString("EXTRA_STATUS")
          val photo =
    arguments?.getString("EXTRA_PHOTO")

          binding.tvName.text = name
          binding.tvSpecies.text = species
          binding.tvStatus.text = status

          Glide.with(this)
              .load(photo)
              .circleCrop()
              .into(binding.imgItemPhoto)
```

```
        return binding.root
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

25. presentation/adapter/FavoriteAdapter.kt

Tabel 25. Source Code Jawaban Soal 1

```
1  package
2  com.example.rickandmortyapi.presentation.adapter
3
4  import android.annotation.SuppressLint
5  import android.view.LayoutInflater
6  import android.view.ViewGroup
7  import androidx.recyclerview.widget.RecyclerView
8  import com.bumptech.glide.Glide
9  import
10 com.example.rickandmortyapi.data.model.CharacterE
11 ntity
12 import
13 com.example.rickandmortyapi.databinding.ItemRowFa
14 voriteBinding
15
16 class FavoriteAdapter(
17     private val items:
18 ArrayList<CharacterEntity>,
19     private val onRemoveClick: (CharacterEntity)
20 -> Unit
```

```
) :
RecyclerView.Adapter<FavoriteAdapter.FavViewHolde
r>() {

    inner class FavViewHolder(val binding:
ItemRowFavoriteBinding) :
            RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent:
ViewGroup, viewType: Int): FavViewHolder {
        val binding =
ItemRowFavoriteBinding.inflate(LayoutInflater.fro
m(parent.context), parent, false)
        return FavViewHolder(binding)
    }

    override fun getItemCount() = items.size

    override fun onBindViewHolder(holder:
FavViewHolder, position: Int) {
        val item = items[position]
        holder.binding.apply {
            itemNameFav.text = item.name
            itemSpeciesFav.text = item.species
            itemAliveFav.text = item.status

            Glide.with(itemImageFav.context)
                .load(item.image)
                .into(itemImageFav)

            btnRemoveFav.setOnClickListener {
                onRemoveClick(item)
```

```
            }
        }
    }

    @SuppressLint("NotifyDataSetChanged")
    fun setData(newData: List<CharacterEntity>) {
        items.clear()
        items.addAll(newData)
        notifyDataSetChanged()
    }
}
```

26. presentation/favorites/FavoriteViewModel.kt

Tabel 26. Source Code Jawaban Soal 1

```
1  package
2  com.example.rickandmortyapi.presentation.favorite
3  s
4
5  import androidx.lifecycle.ViewModel
6  import androidx.lifecycle.viewModelScope
7  import
8  com.example.rickandmortyapi.data.model.CharacterE
9  ntity
10 import
11 com.example.rickandmortyapi.data.repository.Chara
12 cterRepository
13 import kotlinx.coroutines.launch
14
15 class FavoriteViewModel(private val repository:
16 CharacterRepository): ViewModel() {
17     val favorites = repository.favorites
18
```

```
    fun removeFavorite (character:
CharacterEntity) {
        viewModelScope.launch {
            repository.removeFavorite(character)
        }
    }
}
```

27. presentation/utils/FavoriteViewModelFactory.kt

Tabel 27. Source Code Jawaban Soal 1

```
package com.example.rickandmortyapi.utils

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import
com.example.rickandmortyapi.data.repository.Chara
cterRepository
import
com.example.rickandmortyapi.presentation.favorite
s.FavoriteViewModel

class FavoriteViewModelFactory(private val
repository: CharacterRepository):
ViewModelProvider.Factory {

    override fun <T : ViewModel>
create(modelClass: Class<T>): T {
        if
(modelClass.isAssignableFrom(FavoriteViewModel::c
lass.java)) {
            return FavoriteViewModel(repository)
as T
```

```
        }
        throw IllegalArgumentException("Unknown
ViewModel class")
    }
}
```

28. presentation/favorites/FavoriteFragment.kt

Tabel 28. Source Code Jawaban Soal 1

```
1  package
2  com.example.rickandmortyapi.presentation.favorite
3  s
4
5  import android.os.Bundle
6  import androidx.fragment.app.Fragment
7  import android.view.LayoutInflater
8  import android.view.View
9  import android.view.ViewGroup
10 import android.widget.Toast
11 import androidx.lifecycle.ViewModelProvider
12 import
13 com.example.rickandmortyapi.data.repository.Chara
14 cterRepository
15 import
16 com.example.rickandmortyapi.databinding.FragmentF
17 avoriteBinding
18 import
19 com.example.rickandmortyapi.presentation.adapter.
20 FavoriteAdapter
   import
   com.example.rickandmortyapi.utils.FavoriteViewMod
   elFactory
   import
```

```
com.example.rickandmortyapi.utils.Injection

class FavoriteFragment : Fragment() {
    private var _binding:
FragmentFavoriteBinding? = null
    private val binding get() = _binding!!

    private lateinit var viewModel:
FavoriteViewModel
    private lateinit var favoriteAdapter:
FavoriteAdapter

    override fun onCreateView(
        inflater: LayoutInflater, container:
ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        _binding =
FragmentFavoriteBinding.inflate(inflater,
container, false)

        return binding.root
    }

    override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
        super.onViewCreated(view,
savedInstanceState)

        val repository: CharacterRepository =
Injection.provideRepository(requireContext())
        val factory =
```

```
    FavoriteViewModelFactory(repository)
            viewModel = ViewModelProvider(this,
    factory)[FavoriteViewModel::class.java]

            favoriteAdapter =
    FavoriteAdapter(arrayListOf()) {
                viewModel.removeFavorite(it)
                Toast.makeText(requireContext(),
    "${it.name} dihapus dari favorit",
    Toast.LENGTH_SHORT).show()
            }

            // Pasang adapter ke RecyclerView
            binding.rvFavorite.adapter =
    favoriteAdapter

            // Observe data LiveData dari Room

    viewModel.favorites.observe(viewLifecycleOwner) {
    favorites ->
                favoriteAdapter.setData(favorites)
            }
        }
    }
```

29. presentation/MainActivity.kt

Tabel 29. Source Code Jawaban Soal 1

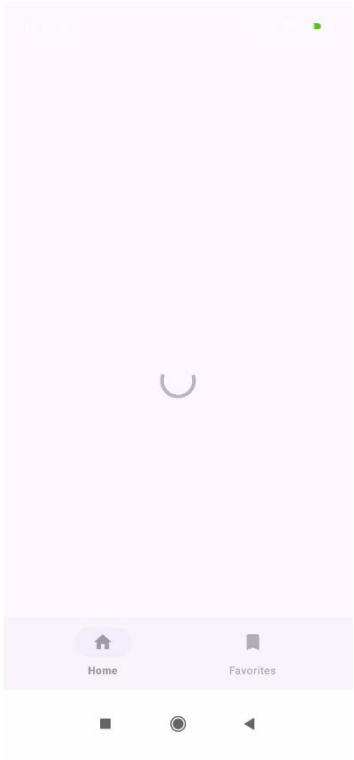
```
1  package com.example.rickandmortyapi.presentation
2
3  import android.os.Bundle
4  import android.util.Log
5  import android.view.Menu
```

```
6  import android.view.MenuItem
7  import androidx.activity.enableEdgeToEdge
8  import androidx.appcompat.app.AppCompatActivity
9  import androidx.fragment.app.Fragment
10 import com.example.rickandmortyapi.R
11 import
12 com.example.rickandmortyapi.databinding.ActivityM
13 ainBinding
14 import
15 com.example.rickandmortyapi.presentation.favorite
16 s.FavoriteFragment
17 import
18 com.example.rickandmortyapi.presentation.home.Hom
19 eFragment
20

   class MainActivity : AppCompatActivity() {
       private lateinit var binding:
   ActivityMainBinding
       val fragHome = HomeFragment()
       val fragFavorites = FavoriteFragment()
       val mFragmentManager = supportFragmentManager
       // active fragment
       var active: Fragment = fragHome

       private lateinit var menu: Menu
       private lateinit var menuItem: MenuItem

       override fun onCreate(savedInstanceState:
   Bundle?) {
           super.onCreate(savedInstanceState)
           enableEdgeToEdge()
           binding =
```

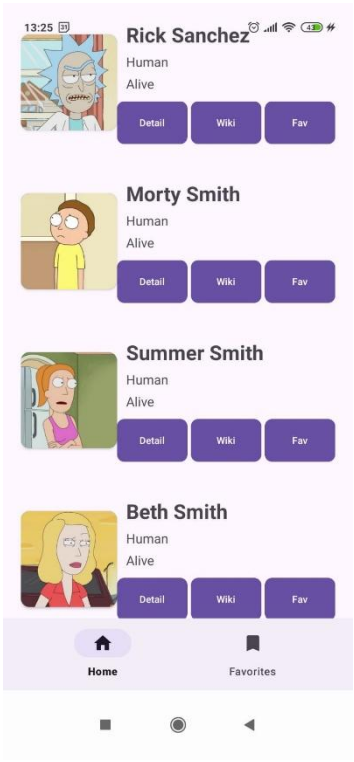
```
ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        setUpNaviBottom()
    }

    private fun setUpNaviBottom () {
        mFragmentManager.beginTransaction().apply
{
            add(R.id.navigation_content,
fragFavorites).hide(fragFavorites)
            add(R.id.navigation_content,
fragHome)
        }.commit()

        active = fragHome

        menu = binding.btnNavigation.menu
        menuItem = menu.getItem(0)
        menuItem.isChecked = true


binding.btnNavigation.setOnNavigationItemSelected
Listener {
            item ->
            when (item.itemId) {
                R.id.navigation_home -> {
                    callFrag(0, fragHome)
                }
                R.id.navigation_fav -> {
                    callFrag(1, fragFavorites)
                }
```

```
            }
            true
        }
    }

    private fun callFrag (i: Int, fragment:
Fragment) {
        if (fragment != active) {
            Log.d("MyFlexibleFragment", "Fragment
Name : " + active::class.java.simpleName)
            menuItem = menu.getItem(i)
            menuItem.isChecked = true

            mFragmentManager.beginTransaction()
                .hide(active)
                .show(fragment)
                .commit()

            active = fragment
        }
    }
}
```

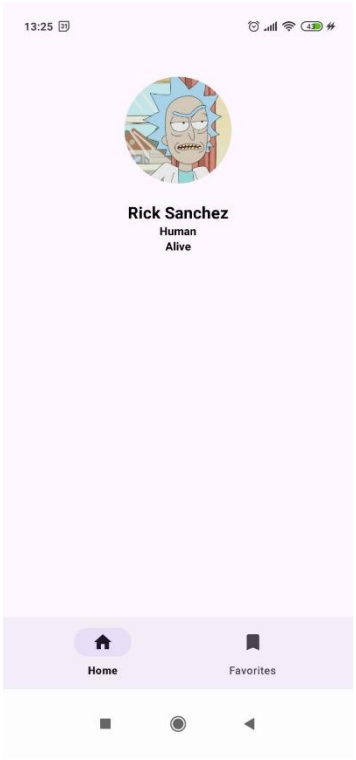
B. Output Program



Gambar 1. Screenshot Output Soal 1

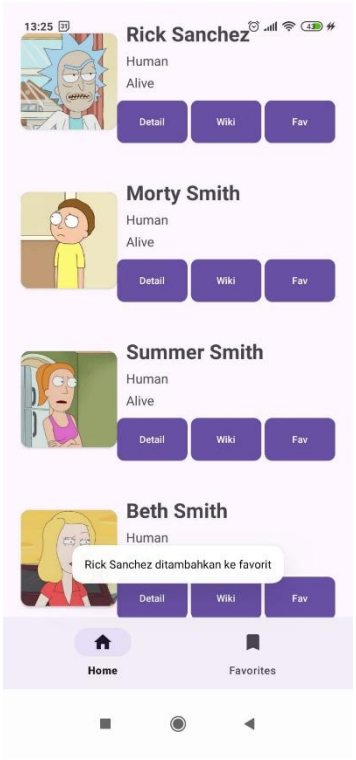


Gambar 2. Screenshot Output Soal 1

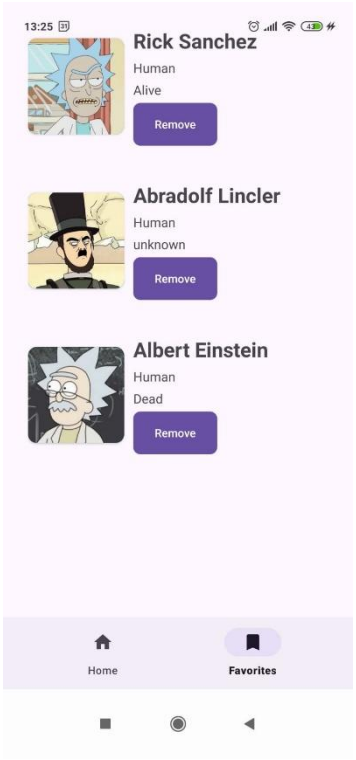


Gambar 3. Screenshot Output Soal 1

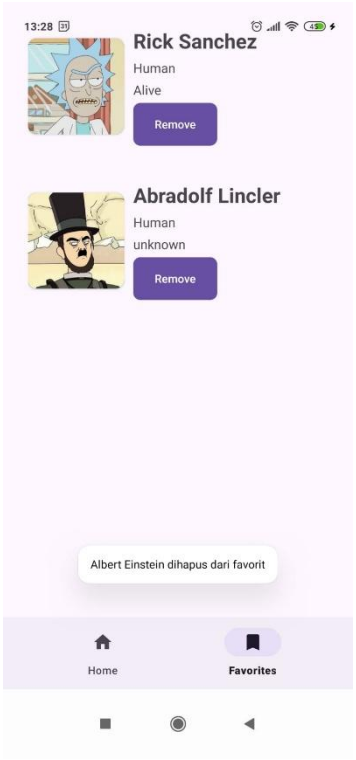
Gambar 4. Screenshot Output Soal 1



Gambar 5. Screenshot Output Soal 1



Gambar 6. Screenshot Output Soal 1



Gambar 7. Screenshot Output Soal 1

C. Pembahasan

Aplikasi yang berisikan daftar karakter grup mahasiswa yang bernama *WarikMaxxing*.

1. item_row_morty.xml

Tampilan RecyclerView dibuat dalam item_character.xml, dengan menggunakan CardView dan ConstraintLayout. Dalam tampilan, terdapat 5 item, gambar, judul, deskripsi singkat, dan dua tombol (*details* dan *contact*)

Layout utama menggunakan CardView dengan tampilan melengkung untuk memberikan tampilan seperti kartu.

a. ConstraintLayout

Layout fleksibel untuk mengatur posisi elemen di dalam CardView.

b. CardView dan ImageView

Membungkus gambar dengan menggunakan CardView.

c. TextView

Menampilkan nama atau julukan deskripsi tentang karakter.

d. MaterialButton (Detail, Wiki, dan Fav)

Menampilkan detail karakter yang berpindah ke DetailFragment ketika tombol detail diklik, kemudian tombol wiki akan mengarahkan ke aplikasi luar, dan tombol fav yang akan menyimpan tiap karakter ke dalam halaman favorite.

2. fragment_home.xml

Tampilan RecyclerView yang telah dibuat dalam item_character.xml akan ditampilkan di dalam fragment_home.xml.

Layout utama menggunakan ConstraintLayout, di dalamnya terdapat RecyclerView.

a. RecylerView

Menampilkan daftar dari layout item_character.xml.

b. ProgressBar

Menampilkan progress bar ketika aplikasi sedang melakukan proses pengambilan data dari API.

c. TextView

Ketika aplikasi terdapat error, akan ditampilkan dalam teks ini.

3. fragment_detail.xml

Tampilan halaman detail, layout utama dibuat dengan ConstraintLayout.

a. ImageView

Menampilkan foto karakter, yang akan dibuat menjadi circleCrop dengan menggunakan Glide.

b. TextView (Nama, Spesies, dan Status)

Menampilkan nama, spesies, dan status tentang karakter.

4. item_row_fav.xml

Tampilan RecyclerView khusus untuk halaman yang menyimpan karakter yang telah difavoritkan, Dibuat dengan menggunakan CardView dan ConstraintLayout. Dalam tampilan, terdapat 5 item, gambar, judul, deskripsi singkat, dan dua tombol (*details* dan *contact*)

Layout utama menggunakan CardView dengan tampilan melengkung untuk memberikan tampilan seperti kartu.

a. ConstraintLayout

Layout fleksibel untuk mengatur posisi elemen di dalam CardView.

b. CardView dan ImageView

Membungkus gambar dengan menggunakan CardView.

c. TextView

Menampilkan nama atau julukan deskripsi tentang karakter.

d. MaterialButton (Remove)

Menampilkan tombol remove yang akan menghapus karakter yang diremove dan menghilangkannya dari halaman favorite.

5. **fragment_fav.xml**

   Tampilan halaman favorite yang berisi karakter-karakter yang telah difavoritkan. Ditampilkan dengan menggunakan RecyclerView yang telah dibuat sebelumnya.

6. **bottom_navigation_menu.xml**

   File yang digunakan sebagai item menu navigasi yang berada di bawah. Menu ini ditampilkan di bagian bawah layar dan memungkinkan pengguna untuk berpindah halaman Home dan Favorites (*fragment*). File ini juga merupakan penghubungan dengan komponen lain yang digunakan oleh MainActivity.

   a. Menu

      Menu merupakan root dari file yang menampung navigasi antar halaman.

   b. Item (Home dan Favorites)

      Kedua item yaitu Home dan Favorites akan ditampilkan di bawah dan ikon menu ditampilkan menggunakan icon gambar yang diambil dari drawable.

7. **activity_main.xml**

   File yang digunakan sebagai tampilan utama, lalu mengatur posisi tiap fragment dan menampilkan navigasi yang berada di bawah layar sebagai navigasi utama aplikasi.

   Layout utama menggunakan RelativeLayout sebagai root yang mengatur posisi relatif tiap elemen di dalamnya. Lalu menggunakan FrameLayout sebagai container tiap fragment (Home dan Favorites). Kemudian yang terakhir, menampilkan komponen dari MaterialDesign untuk tampilan navigasi bawah.

8. **CharacterDto.kt**

   Data transfer object merupakan file yang merepresentasikan struktur data karakter yang dikembalikan oleh API. Dto akan menampung isi data dari JSON yang dikirim oleh API.

9. **CharacterResponse.kt**

   File yang menampung isi *response* dari API yang berisi daftar karakter. Tiap response tersebut akan disimpan ke dalam variabel *results* yang akan digunakan dalam mengambil data responnya.

10. **CharacterEntity.kt**

    Entity untuk Room Database yang menyimpan data karakter favorit ke penyimpanan lokal. Anotasi @Entity memberitahu Room bahwa kelas ini adalah entitas tabel dalam Database.

11. **CharacterDao.kt**

    Data access object adalah antarmuka yang digunakan Room untuk mengakses database lokal yang sebelumnya telah disimpan. Di dalamnya terdapat 3 anotasi:

    1. @Query untuk mengambil seluruh data favorit
    2. @Insert untuk menambah karakter ke favorit
    3. @Delete untuk menghapus karakter dari favorit

12. **AppDatabase.kt**

    Database Room yang menyediakan akses ke DAO. File ini akan mendefinisikan daftar-daftar entity yang akan digunakan dengan anotasi @Database. File ini juga menerapkan *singleton pattern* agar menghindari inisialisasi database yang berulang.

13. CharacterRepository.kt

Repository yang menjadi penghubung antara API, database, dan ViewModel. File ini akan mengelola logika data dan sumber data dari API. Di dalamnya terdapat 3 fungsi:

1. getFavorites untuk mendapatkan semua karakter yang difavoritkan
2. addFavorite untuk menambahkan karakter ke favorite
3. removeFavorite untuk menghapus karakter yang telah difavoritkan

14. CharacterViewModel.kt

ViewModel yang mengatur data karakter dari repository dan menyiapkan data-datanya agar ditampilkan di UI. StateFlow digunakan untuk mengkonsumsi UI dan fungsi *fetchCharacters()* akan dipanggil saat viewModel diinisialisasi untuk mengambil datanya dari repository.

15. CharacterRepositoryInstance.kt

File untuk *dependencty injection*, dilakukan secara manual untuk menyediakan instance dari CharacterRepository. File ini menginisialisasi AppDatabse dan ApiService agar dapat digunakan oleh CharacterRepository.

16. Resource.kt

File yang membungkus tiap status dari respon-respon yang diterima, yaitu sukses, loading, dan error. Tiap status ini akan disimpan dan digunakan pada viewModel dan UI. Respon pada UI akan menampilkan progress bar, menampilkan data, atau pesan error.

17. HomeViewModelFactory.kt

Factory class untuk membuat instance HomeViewModel dengan menggunakan parameter repository.

18. Injection.kt

File yang menyediakan instance dari CharacterRepository kemudian mengambil CharacterDao dari database dan menyiapkannya agar dapat dipakai ke ViewModel.

19. HomeViewModel.kt

Mengambil data dan mennyimpan tiap data dari API ke dalam StateFlow yang akan diambil oleh UI.

20. HomeFragment.kt

Halaman yang menampilkan daftar-daftar karakter dari API menggunakan RecyclerView dan datanya diambil dari CharViewModel.kt.

21. DetailFragment.kt

Halaman yang dapat diakses ketika tombol detail diklik, yang akan menghubungkan ke halaman detail yang berisi detail lengkap dari karakter.

22. FavoriteAdapter.kt

Adapter untuk menampilkan data karakter yang difavoritkan. Datanya akan diambil dari database lokal menggunakan RecyclerView dan tiap datanya akan ditampilkan ke fragment FavoriteFragment.

23. FavoriteViewModel.kt

ViewModel untuk mengatur data-data dari karakter yang difavoritkan yang diambil dari Room. Terdapat dua fungsi:

1. loadFavorites untuk mengambil data karakter favorit dari Room
2. removeFavorite untuk menhapus data karakter favorit dari Room

24. FavoriteViewModelFactory.kt

Factory untuk membuat FavoriteViewModel dengan menggunakan parameter Repository. File ini sama seperti HomeViewModelFactory yang digunakan untuk menyediakan dependensi ke FavoriteViewModel.

25. FavoriteFragment.kt

Halaman yang akan menampilkan daftar-daftar karakter favorit yang telah ditandai dengan menggunakan adapter FavoriteAdapter. Tiap karakter ditampilkan dengan menggunakan RecyclerView dan terdapat tombol remove untuk menghapus karakter yang telah difavoritkan.

26. MainActivity.kt

File yang mengelola seluruh fragment dan mengelola navigasi antar fragmentnya. File ini akan menyediakan navigasi ke halaman-halaman home, detail, dan favorite.

Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat http://github.com/akhmadCh/Pemrograman-Mobile/tree/master/Modul%205